

CHAPTER 1

Introduction to Software Quality Assurance

Coverage

12 Sections

Course

Software Engineering

Deck

52 Slides

Topics

SQA · Frameworks · Trends

Chapter 1 — Table of Contents

1.1 Introduction to Software Quality Assurance

1.2 Definition of Software Quality

1.3 Definition of Software Quality Assurance

1.4 Objectives of Software Quality Assurance

1.5 Importance of Software Quality Assurance

1.6 Evolution of Software Quality Assurance

1.7 Software Quality Assurance Frameworks

1.8 Roles and Responsibilities in SQA

1.9 Challenges in Software Quality Assurance

1.10 Benefits of Effective Software Quality Assurance

1.11 Future Trends in Software Quality Assurance

1.12 Chapter Summary

1.1

Introduction to Software Quality Assurance

Setting the stage — why quality matters in software.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.1

Why Software Quality Assurance?

Software is embedded in nearly every aspect of modern life — from healthcare to banking, aviation to social media. Failures are costly, dangerous, and reputational.

Ariane 5 Rocket (1996)

Software reuse error caused explosion. \$500M loss in 40 seconds.

Therac-25 (1985–87)

Radiation therapy machine software defects caused patient deaths.

Knight Capital (2012)

Faulty trading software lost \$440M in 45 minutes.

Boeing 737 MAX (2018)

MCAS software defect contributed to two fatal crashes — 346 lives.

NHS Wannacry (2017)

Unpatched systems caused £92M damage; 19,000 appointments cancelled.

Facebook Down (2021)

Configuration error caused 6-hour global outage; \$60M revenue loss.



These failures drove the development of structured Software Quality Assurance (SQA) practices, standards, and frameworks.

Section 1.1

The Software Crisis & Birth of SQA

The 'Software Crisis' (NATO Conference, 1968) revealed systematic problems with software development — triggering the need for quality assurance:

! Software projects consistently delivered late and over budget

! Software was unreliable — frequent failures in production

! Software was difficult to maintain and evolve

! There were no systematic methods for developing software

! No standards or processes to ensure consistent quality

! Testing was an afterthought, not a planned activity

Key SQA Milestones:

1968 NATO Software Crisis Conference

1972 Dijkstra — structured programming

1979 IEEE Software Engineering Standards

1987 ISO 9000 Quality Standards

1991 CMM (Capability Maturity Model)

1996 ISO/IEC 9126 Software Quality

2001 Agile Manifesto published

2011 ISO/IEC 25010 SQuaRE

1.2

Definition of Software Quality

What constitutes quality in a software product?

Section 1.2

Software Quality — Definitions

Software quality is defined from multiple authoritative perspectives:

IEEE Std 610.12

The degree to which a system, component, or process meets specified requirements; meets customer or user needs or expectations.

ISO 8402 / ISO 9000

The totality of characteristics of an entity that bear on its ability to satisfy stated and implied needs.

Philip Crosby (1979)

"Quality is conformance to requirements." Quality means meeting the specifications — not exceeding them unnecessarily.

Joseph Juran (1988)

"Quality is fitness for use." A product has quality if it successfully serves the user's purpose during actual usage.

Roger Pressman

Software quality is defined as conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics expected of professionally developed software.

Section 1.2

Three Dimensions of Software Quality



Product Quality

Quality of the software product itself

- Correctness — does it do what it should?
- Reliability — does it work without failure?
- Efficiency — does it use resources well?
- Usability — is it easy to use?
- Maintainability — easy to change?
- Portability — works across environments?



Process Quality

Quality of the development process

- Defined and documented processes
- Consistent and repeatable methods
- Measurable and improvable practices
- Standards compliance (ISO, CMMI)
- Peer reviews and inspections
- Configuration management



Quality in Use

Quality as experienced by the user

- Effectiveness — achieves user goals?
- Efficiency — resources expended?
- Satisfaction — user comfort/happiness?
- Safety — freedom from harm?
- Context coverage — all scenarios?



SQA addresses all three dimensions — ensuring the product, process, AND user experience meet defined quality standards.

1.3

Definition of Software Quality Assurance

SQA — a systematic approach to quality.

Section 1.3

What is Software Quality Assurance (SQA)?

Software Quality Assurance (SQA)

A planned and systematic set of activities necessary to provide adequate confidence that software processes and products conform to established technical requirements and management needs. SQA encompasses the entire software development life cycle.

Key Distinctions:

SQA vs Quality Control (QC)

SQA is PROACTIVE (prevents defects). QC is REACTIVE (finds defects after creation).

SQA vs Testing

SQA encompasses the entire SDLC. Testing is a single phase within SQA.

SQA vs Verification

Verification checks if we built the product RIGHT. Validation checks if we built the RIGHT product.

Section 1.3

SQA Activities & Scope

SQA activities span the entire Software Development Life Cycle (SDLC):

Requirements

- Review requirements for clarity
- Ensure testability of requirements
- Validate completeness

Design

- Design reviews and inspections
- Architecture quality assessment
- Design standard compliance

Coding

- Code reviews and walkthroughs
- Coding standard enforcement
- Static analysis tools

Testing

- Test planning and design
- Test execution monitoring
- Defect tracking and analysis

Deployment

- Release criteria verification
- Production readiness review
- Post-deployment monitoring

Maintenance

- Change impact assessment
- Regression testing
- Process improvement

1.4

Objectives of Software Quality Assurance

What SQA aims to achieve.

Section 1.4

Primary Objectives of SQA

SQA has six primary objectives that guide its activities throughout the SDLC:

01

Ensure Standards Compliance

Verify that software products and processes adhere to defined standards, guidelines, and procedures (IEEE, ISO, CMMI).

02

Prevent Defects

Identify and eliminate defects early in the development cycle before they propagate to later, more expensive phases.

03

Improve Process Quality

Evaluate and improve the software development and maintenance processes to increase efficiency and effectiveness.

04

Verify Requirements Fulfilment

Ensure that all specified user and system requirements are correctly implemented in the delivered software.

05

Provide Objective Evidence

Supply management with objective data and metrics about software quality to support informed decision-making.

06

Ensure Customer Satisfaction

Guarantee that the final product meets user expectations, is reliable, and fit for the intended purpose of use.

Section 1.4

SQA Objectives vs. Activities Mapping

How SQA activities map to objectives:

SQA Objective	Key Activities	Expected Outcome
Standards Compliance	Audits · Inspections · Reviews	Consistent quality across projects
Defect Prevention	Reviews · Static analysis · Walkthroughs	Fewer defects, lower fix costs
Process Improvement	Metrics collection · Analysis · PDCA	Faster, cheaper delivery
Requirements Verification	Traceability matrix · Prototype review	Correct functionality delivered
Objective Evidence	Metrics · Reports · Dashboards	Informed management decisions
Customer Satisfaction	UAT · Acceptance testing · Feedback	Fit-for-purpose product delivered

 The overarching objective of SQA is to give management confidence that the software development process produces a quality product. SQA does NOT do the work — it monitors, measures, and reports.

1.5

Importance of Software Quality Assurance

Why SQA is essential in modern software engineering.

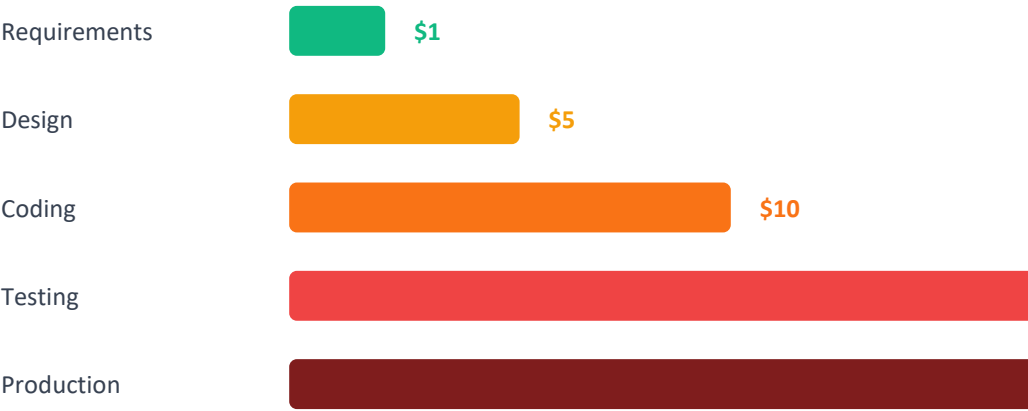
Chapter 1 — Introduction to Software Quality Assurance

Section 1.5

The Cost of Poor Quality

The cost of poor software quality is measured across multiple dimensions:

Cost to Fix a Defect (Relative)



Categories of Quality Costs:

Prevention Costs

Training, process planning, reviews, tools

Appraisal Costs

Testing, inspections, audits, evaluations

Internal Failure

Rework, retesting, scrap within project

External Failure

Warranty, support, recalls, reputation loss

🔗 IBM Study: A defect found in production costs 40–1000× more to fix than one found during requirements.

Section 1.5

Why SQA is Essential — Key Reasons



Safety & Human Life

In safety-critical systems (aviation, medical, nuclear), software defects can directly cause injury or death. SQA is mandatory in these domains.



Financial Protection

Software failures cost billions annually. Effective SQA dramatically reduces costs of defect rectification, legal liability, and business disruption.



Legal & Regulatory Compliance

Many industries (banking, healthcare, defence) require compliance with software quality standards. Non-compliance risks fines and operational shutdown.



Competitive Advantage

High-quality software builds customer trust, brand reputation, and market share. Poor quality drives customers to competitors.



Project Success Rates

Structured SQA significantly increases the probability of projects being delivered on time, within budget, and meeting requirements.



Maintainability & Longevity

Quality software is easier to maintain, extend, and adapt. It reduces technical debt and allows organisations to evolve systems over time.

1.6

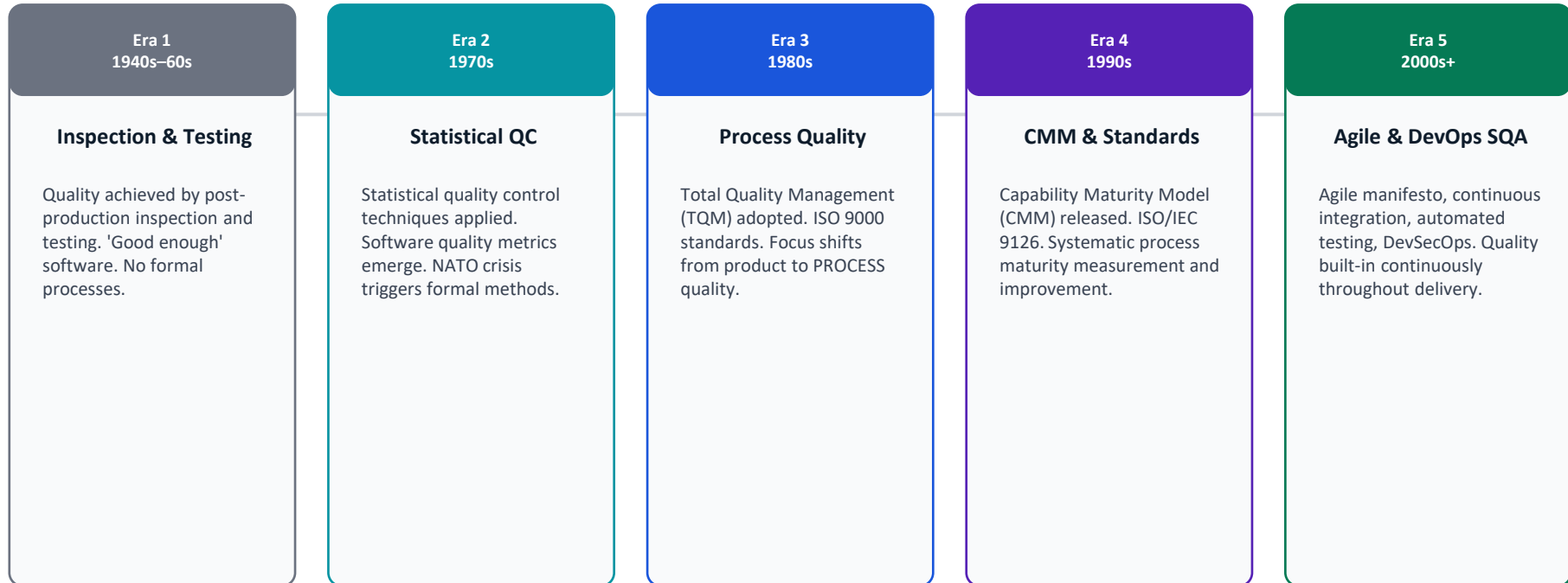
Evolution of Software Quality Assurance

From testing to TQM — the history of SQA.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.6

Evolution of SQA — Five Eras



Section 1.6

Key SQA Historical Milestones

1949	Alan Turing writes first checklist for software testing	1957	First formal software inspection proposed by Harlan Mills
1968	NATO Software Engineering Conference — 'Software Crisis' coined	1972	Dijkstra introduces structured programming concepts
1976	Michael Fagan at IBM introduces formal code inspections	1979	Philip Crosby publishes 'Quality is Free' — zero defects concept
1987	ISO 9000 quality management standards published	1987	Watts Humphrey releases first Capability Maturity Model (CMM)
1991	SEI releases CMM v1.0 formally	1994	IEEE publishes Software Quality Assurance Plan standard (IEEE 730)
1995	Cleanroom software engineering method formalised	1999	Extreme Programming (XP) introduces test-first development
2001	Agile Manifesto — quality through collaboration and iteration	2006	CMMI v1.2 released — integrates SW-CMM, SE-CMM, and IPPD
2011	ISO/IEC 25010 (SQuaRE) replaces ISO 9126	2018	DevSecOps emerges — security built into CI/CD pipelines

1.7

Software Quality Assurance Frameworks

Standards, models, and methodologies for SQA.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.7

Major SQA Frameworks Overview

Multiple frameworks guide SQA implementation — each with different scope, focus, and applicability:

**CMMI
Capability Maturity Model
Integration**

2002+

By: SEI / ISACA

Process maturity and improvement across 5 levels

ISO 9000 / ISO 9001

1987+

By: ISO

Quality management systems for organisations

**ISO/IEC 25010
(SQuaRE)**

2011

By: ISO/IEC

Product quality characteristics and measurement

**IEEE 730
SQA Plans Standard**

1979+

By: IEEE

Requirements for SQA planning and documentation

**Six Sigma
DMAIC / DFSS**

1986+

By: Motorola / GE

Statistical defect reduction — 3.4 defects/million

**ISTQB Testing
Framework**

2002+

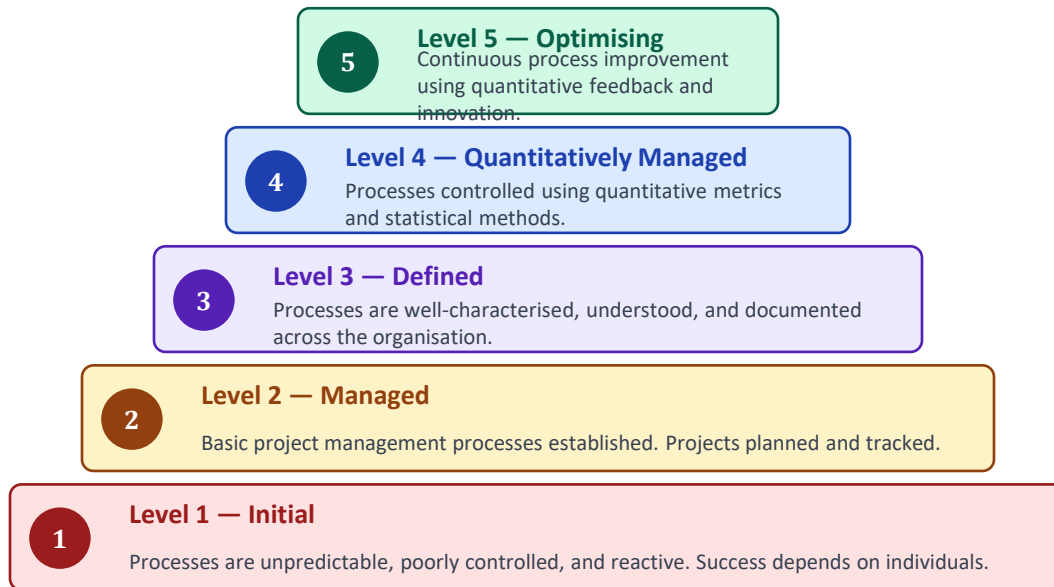
By: ISTQB

International testing qualifications and standards

Section 1.7

CMMI — Capability Maturity Model Integration

CMMI (Capability Maturity Model Integration) is the most widely used SQA process improvement framework:



✂ Most large organisations target CMMI Level 3. Defence contractors often require Level 3–4. NASA and high-reliability systems target Level 5.

Section 1.7

IEEE 730 — SQA Plan Standard

IEEE Std 730 specifies the format and content of a Software Quality Assurance Plan (SQAP):

Section	Name	Content
Section 1	Purpose	What the software product is and what the SQA plan covers
Section 2	Reference Documents	All standards, guidelines, and documents referenced
Section 3	Management	Organisation, tasks, and responsibilities for SQA
Section 4	Documentation	Documents required (SRS, SDD, STP, STR, SDR)
Section 5	Standards, Practices, Conventions	Coding standards, design methods, documentation rules
Section 6	Reviews and Audits	Types: SRR, PDR, CDR, FCA, PCA, process audits
Section 7	Test	Test plan, test cases, procedures, regression testing
Section 8	Problem Reporting & Corrective Action	Defect tracking, reporting procedures, corrective actions
Section 9	Tools, Techniques, Methodologies	CASE tools, static analysis, formal methods used
Section 10	Code Control	Configuration management, version control, baseline
Section 11	Media Control	Handling of source code, deliverables, backups
Section	Supplier Control	Third-party software vendor quality requirements

1.8

Roles and Responsibilities in SQA

Who does what in software quality assurance.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.8

SQA Team Roles & Responsibilities

SQA Manager

- Plan and manage the overall SQA programme
- Define SQA policies and procedures
- Report quality status to senior management
- Manage SQA team and resources
- Coordinate audits and reviews

SQA Engineer / Analyst

- Conduct process and product audits
- Review and inspect work products
- Track and report defects
- Verify standards compliance
- Maintain quality records

Quality Auditor

- Conduct independent process audits
- Assess adherence to defined processes
- Identify non-conformances
- Report findings objectively
- Follow up on corrective actions

Test Engineer

- Design and execute test cases
- Report and track defects
- Perform regression testing
- Develop test automation
- Maintain test documentation

Configuration Manager

- Maintain version control of all artefacts
- Manage baselines and releases
- Control changes to configuration items
- Audit configuration management
- Report configuration status

Project Manager

- Ensure SQA plan is developed
- Allocate resources for SQA activities
- Review and act on SQA reports
- Remove barriers to quality
- Champion quality culture

Section 1.8

SQA Organisation Structure

SQA teams can be organised in three structures — each with trade-offs:

Independent SQA Group

Separate from development team

✓ Advantages:

- Objective and unbiased assessments
- Reports directly to senior management
- Clear separation of concerns

✗ Disadvantages:

- Can create adversarial culture
- Communication overhead
- Higher organisational cost

Embedded QA Team

QA within the development team

✓ Advantages:

- Close collaboration with devs
- Fast feedback cycles
- Better team integration

✗ Disadvantages:

- Risk of bias / groupthink
- Manager conflict of interest
- Less independence

Virtual / Matrix QA

QA shared across multiple projects

✓ Advantages:

- Efficient resource utilisation
- Consistent standards across projects
- Expertise sharing

✗ Disadvantages:

- Split loyalties and priorities
- Communication complexity
- Harder to manage

1.9

Challenges in Software Quality Assurance

Obstacles and difficulties in implementing SQA.

Section 1.9

Technical Challenges in SQA



Complexity of Modern Software

Modern systems involve millions of lines of code, microservices, APIs, distributed architectures, and AI components — making comprehensive quality assurance extremely difficult.



Rapidly Changing Requirements

Agile environments have frequently changing requirements. Maintaining quality while adapting to change is a constant challenge for SQA teams.



Testing Incompleteness

It is theoretically impossible to test all possible inputs and scenarios. SQA must risk-prioritise testing while accepting that 100% coverage is unachievable.



Legacy System Quality

Old systems with poor documentation, high technical debt, and no automated tests are extremely difficult to quality-assure during maintenance and evolution.



Emerging Technology Gaps

AI/ML, IoT, blockchain, quantum computing present new quality challenges with no established SQA standards, metrics, or assessment methods yet.



Tool & Automation Limitations

Test automation requires significant investment. Not all scenarios can be automated. Tool selection and maintenance adds overhead to SQA activities.

Section 1.9

Organisational & Human Challenges

Management Resistance

SQA is often viewed as overhead. Management may resist investment in SQA activities due to short-term cost pressures and schedule pressure.

💡 *Build business case using cost-of-poor-quality data. Show ROI.*

Developer-QA Conflict

Tension between developers who create and QA who critique. Cultural resistance to external quality checks creates friction and reduces effectiveness.

💡 *Promote quality culture. Make QA collaborative, not adversarial.*

Insufficient Resources

SQA requires skilled people, time, tools, and budget. Under-resourced SQA teams cannot adequately cover all software artefacts and processes.

💡 *Risk-based prioritisation. Automate to extend coverage with less effort.*

Unclear Quality Criteria

Without clear, measurable definitions of 'done' and 'quality', SQA teams cannot objectively assess whether quality standards have been met.

💡 *Define clear acceptance criteria, quality gates, and DoD upfront.*

Time-to-Market Pressure

Business pressure to release quickly leads to shortcuts in quality activities — reduced reviews, insufficient testing, deferred defect fixing.

💡 *Shift-left testing. Automate. Build quality in from the start.*

Skills Shortage

SQA requires diverse skills: testing, process, tools, domain knowledge, analytical skills. Finding and retaining skilled SQA professionals is challenging.

💡 *Invest in training, certification (ISTQB), and career development.*

1.10

Benefits of Effective Software Quality Assurance

The measurable value SQA delivers to organisations.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.10

Tangible Benefits of SQA



Reduced Development Costs

Early defect detection reduces rework costs. IBM studies show defects fixed in requirements cost 1× vs 100× in production.



Improved Reliability

Systematic testing and review leads to fewer failures in production. Higher availability and uptime for critical systems.



Higher Customer Satisfaction

Quality software meets user needs, is easy to use, and performs reliably — leading to customer loyalty and repeat business.



Faster Time to Market

Paradoxically, SQA speeds delivery. Fewer defects mean less rework. Automated testing speeds release cycles significantly.



Regulatory Compliance

SQA ensures adherence to industry standards (ISO, FDA, FAA), avoiding fines, recalls, and operational shutdowns.



Reduced Maintenance Costs

High-quality, well-documented, tested software is cheaper to maintain, extend, and evolve over its operational lifetime.



Brand Reputation

Consistently delivering quality software builds brand reputation, trust, and competitive differentiation in the marketplace.



Risk Reduction

SQA identifies risks early when they are cheapest to mitigate. Reduces probability of catastrophic failures in production.

Section 1.10

SQA ROI — Key Statistics & Evidence

Evidence-based benefits of implementing effective SQA:

40%

Average cost reduction in defect fixing when using code reviews vs. testing alone.

Source: IBM / Fagan (1976)

100×

Cost multiplier — fixing a defect in production vs. catching it in requirements phase.

Source: Barry Boehm (1981)

\$2.41T

Annual cost of poor software quality in the USA alone (2022 estimate).

Source: Consortium for IT Quality

3–5×

ROI from investing in software quality assurance activities.

Source: SEI Research (2001)

75%

Of software projects without proper QA processes fail or are significantly challenged.

Source: Standish CHAOS Report

50%

Reduction in test cycles achieved through automated regression testing.

Source: ISTQB Research

1.1.1 Future Trends in Software Quality Assurance

Where SQA is heading in the next decade.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.11

Emerging Trends Reshaping SQA

AI & Machine Learning in SQA

AI generates test cases automatically, predicts defect-prone code areas, performs intelligent test selection, and enables self-healing test automation. ML models trained on historical defect data guide testing priorities.

DevSecOps Integration

Security quality assurance embedded into the CI/CD pipeline. Automated security scanning (SAST, DAST, SCA) at every commit. Security as code. Compliance automation for regulatory requirements.

Quality Assurance for AI Systems

New frameworks for testing AI/ML models: data quality, model fairness, explainability testing, adversarial testing, bias detection, model drift monitoring in production.

Shift-Left & Shift-Right Testing

Shift-left: testing earlier in development (TDD, BDD). Shift-right: testing in production (canary releases, A/B testing, chaos engineering, observability). Quality throughout the pipeline.

Continuous Quality Engineering

Quality gates at every pipeline stage. Automated quality metrics dashboards. Real-time quality feedback to developers. Quality as a continuous measurement, not a phase-end gate.

Low-Code / No-Code Testing

Citizen testing using no-code test platforms. Business users create and run acceptance tests without coding. AI-assisted test creation reduces dependency on technical testers.

Section 1.11

SQA in the Age of AI & Automation

How Artificial Intelligence is transforming Software Quality Assurance:

Test Generation

Now: Manual test case writing

→ AI generates test cases from requirements and user stories automatically

Defect Prediction

Now: Reactive — find defects after creation

→ ML models predict which code modules are defect-prone before testing begins

Visual Testing

Now: Manual UI comparison

→ AI compares screenshots, detects visual regressions across devices/browsers automatically

Test Maintenance

Now: Manual test script updates after UI changes

→ Self-healing tests automatically adapt to minor UI changes without human intervention

Performance Testing

Now: Manual load test scripting and analysis

→ AI analyses production traffic patterns to generate realistic load tests dynamically

Root Cause Analysis

Now: Manual log analysis and debugging

→ AI correlates anomalies across logs, metrics, traces to pinpoint root causes in seconds



1.12

Chapter Summary

Key takeaways from Chapter 1.

Chapter 1 — Introduction to Software Quality Assurance

Section 1.12

Chapter 1 — Key Concepts Summary

● Software Quality:	Conformance to requirements + fitness for use + satisfying stated and implied needs. Multi-dimensional: product, process, and quality in use.
● SQA Definition:	A planned, systematic set of activities providing confidence that software processes and products conform to established requirements and standards.
● SQA vs QC vs Testing:	SQA = proactive/process-wide. QC = reactive/product-focused. Testing = one phase within SQA. All are complementary, not interchangeable.
● SQA Objectives:	Standards compliance · Defect prevention · Process improvement · Requirements verification · Objective evidence · Customer satisfaction.
● Software Crisis (1968):	Triggered the birth of SQA. Revealed systematic problems: late delivery, unreliability, poor maintainability, no formal processes.
● Cost of Quality:	Prevention < Appraisal < Internal Failure < External Failure. Fix defects early — 1× vs 100× cost in production. Every \$1 SQA investment returns \$3–5.
● SQA Frameworks:	CMMI (5 maturity levels) · ISO 9001 (QMS) · ISO/IEC 25010 (product quality) · IEEE 730 (SQA plans) · Six Sigma (defect reduction).
● CMMI Levels:	Level 1 Initial → Level 2 Managed → Level 3 Defined → Level 4 Quantitatively Managed → Level 5 Optimising.
● SQA Roles:	SQA Manager · SQA Engineer · Quality Auditor · Test Engineer · Configuration Manager · Project Manager — each with defined responsibilities.
● SQA Challenges:	Software complexity · Changing requirements · Legacy systems · Management resistance · Time pressure · AI/emerging tech gaps.
● SQA Benefits:	Reduced costs · Higher reliability · Customer satisfaction · Regulatory compliance · Risk reduction · Brand reputation.
● Future Trends:	AI/ML in testing · Shift-left/right · DevSecOps · Continuous quality · AI system testing · Low-code test automation.

Key Terms & Definitions

Software Quality	The degree to which software meets requirements, satisfies user needs, and conforms to established standards.
Software Quality Assurance (SQA)	Planned, systematic activities providing confidence that software processes and products conform to requirements.
Quality Control (QC)	Reactive process that focuses on finding defects in completed software artefacts through inspection and testing.
Verification	Confirming that a software product correctly implements its specification — 'Are we building the product right?'
Validation	Confirming that the software product meets user needs and requirements — 'Are we building the right product?'
CMMI	Capability Maturity Model Integration — five-level framework for assessing and improving software process maturity.
SQAP	Software Quality Assurance Plan — document specifying SQA activities, standards, and procedures for a project.
Defect	A deviation from the specified requirements or expected behaviour in a software product.
Audit	Independent examination of a software product or process to determine compliance with established standards.
Configuration Management	Systematic control of changes to the configuration of a software product throughout the project lifecycle.
IEEE 730	IEEE Standard for Software Quality Assurance Plans — defines required content and structure of SQAPs.
Six Sigma	Quality improvement methodology targeting 3.4 defects per million opportunities (DPMO).
TQM	Total Quality Management — organisation-wide approach to integrating quality improvement in all functions.

Review & Practice Questions

-
- Q1** Define Software Quality Assurance (SQA). How does it differ from Quality Control (QC) and software testing?
- Q2** Explain what is meant by the 'Software Crisis' (1968). What problems did it reveal, and how did it contribute to the development of SQA?
- Q3** List and explain the six primary objectives of Software Quality Assurance with examples for each.
- Q4** Why is Software Quality Assurance important? Discuss at least five reasons with supporting evidence or examples.
- Q5** Describe the five levels of the Capability Maturity Model Integration (CMMI). What characterises each level?
- Q6** Explain the three perspectives of software quality: Product Quality, Process Quality, and Quality in Use.
- Q7** What is the IEEE 730 standard? List and describe the twelve sections of a Software Quality Assurance Plan (SQAP).
- Q8** Describe the five eras in the evolution of Software Quality Assurance from the 1940s to the present day.
- Q9** Identify four major challenges in implementing Software Quality Assurance. For each challenge, propose a solution.
- Q10** Explain the concept of 'Cost of Quality' in software engineering. What are the four categories of quality costs?
- Q11** Compare and contrast the three organisational structures for SQA teams. What are the advantages and disadvantages of each?
- Q12** Discuss three future trends in Software Quality Assurance. How will AI and machine learning change the role of SQA engineers?

SQA Quick Reference Card — Chapter 1

Print this slide and keep it at your desk

SQA Definition

Planned, systematic activities
Confidence in process & product
Spans entire SDLC
Proactive, not reactive
Reports to management
NOT the same as testing

SQA Objectives

Standards compliance
Prevent defects
Improve process
Verify requirements
Provide evidence
Ensure satisfaction

CMMI Levels

1 — Initial (chaotic)
2 — Managed (basic PM)
3 — Defined (documented)
4 — Quantitatively Managed
5 — Optimising (continuous)
Target: Level 3+

Cost of Quality

Prevention (cheapest)
Appraisal (testing)
Internal failure (rework)
External failure (most costly)
Fix in requirements = 1×
Fix in production = 100×

SQA Frameworks

CMMI (process maturity)
ISO 9001 (quality mgmt)
ISO/IEC 25010 (product)
IEEE 730 (SQA plans)
Six Sigma (defect reduction)
ISTQB (testing standard)

Future Trends

AI/ML test generation
Shift-left testing (TDD)
DevSecOps integration
Continuous quality
AI system testing
No-code test automation

SQA vs. QC vs. Testing — Detailed Comparison

Three commonly confused concepts — clearly distinguished:

Aspect	SQA	Quality Control (QC)	Testing
Focus	Process quality	Product quality	Finding defects
Approach	Proactive (prevent)	Reactive (detect)	Reactive (detect)
When applied	Entire SDLC	After product creation	During/after development
Responsibility	Whole team + SQA	QA/QC team	Test team
Goal	Improve process	Ensure product quality	Find defects in product
Output	Process reports, metrics	Corrected product	Defect reports
Examples	Audits, reviews, standards	Inspection, review of output	Unit, integration, UAT testing
Relationship to defects	Prevent defects occurring	Identify defects in product	Expose defects through execution
Standards	IEEE 730, ISO 9001, CMMI	ISO 9001, industry standards	IEEE 829, ISTQB standards
Mindset	Quality culture and process	Conformance verification	Destructive — find failures

Exam Tips — Chapter 1

Key areas to focus on for examinations:

Definitions (Must Know)

- SQA exact definition — 'planned and systematic'
- Difference between SQA, QC, and Testing
- Three perspectives: verification vs validation
- Quality definitions from IEEE, ISO, Crosby, Juran

SQA Objectives (Memorise 6)

- Standards compliance, Defect prevention, Process improvement
- Requirements verification, Objective evidence, Customer satisfaction
- Be able to give examples for each objective
- Explain the 'objective evidence' objective clearly

CMMI Levels (Must Know)

- All 5 levels with names and characteristics
- Level 1 = Initial (chaotic) to Level 5 = Optimising
- Which level most organisations target (Level 3)
- What distinguishes each level

Common Exam Questions

- Compare SQA with QC and Testing (table format)
- Explain cost of quality with cost multiplier data
- List IEEE 730 SQAP sections
- Describe evolution of SQA in eras

References & Further Reading

- **Pressman, R.S. & Maxim, B.R. (2019)**
Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- **Sommerville, I. (2016)**
Software Engineering (10th ed.). Pearson Education Limited.
- **Crosby, P.B. (1979)**
Quality is Free: The Art of Making Quality Certain. McGraw-Hill.
- **Juran, J.M. (1988)**
Juran's Quality Control Handbook (4th ed.). McGraw-Hill.
- **IEEE Std 730-2014 (2014)**
IEEE Standard for Software Quality Assurance Processes. IEEE.
- **ISO/IEC 25010:2011 (2011)**
Systems and Software Engineering — SQuaRE — System and Software Quality Models. ISO/IEC.
- **Humphrey, W.S. (1989)**
Managing the Software Process. Addison-Wesley Professional.
- **SEI / CMMI Institute (2010)**
CMMI for Development, Version 1.3. Software Engineering Institute, Carnegie Mellon.
- **Fagan, M.E. (1976)**
"Design and Code Inspections to Reduce Errors in Program Development." IBM Systems Journal, 15(3), 182–211.
- **Boehm, B.W. (1981)**
Software Engineering Economics. Prentice-Hall.

SQA Activities Across the SDLC

SQA activities are distributed across all phases of the SDLC — not just testing:

Planning

- Develop SQAP
- Define quality goals
- Select standards
- Risk assessment

Requirements

- Requirements review
- Traceability matrix
- Testability review
- Stakeholder sign-off

Design

- Design inspection
- Architecture review
- Interface review
- Design standards check

Implementation

- Code reviews
- Static analysis
- Coding standards
- Unit test coverage

Testing

- Test plan review
- Test execution
- Defect tracking

Deployment

- Release review
- Acceptance criteria
- Deployment checklist

Maintenance

- Change impact analysis
- Regression testing
- Process metrics

The Five Ws of Software Quality Assurance

What?

SQA is a planned and systematic set of activities to provide adequate confidence that software processes and products conform to established technical requirements and management needs.

Why?

Because software failures are costly (lives, money, reputation). Early defect detection is 40-100× cheaper. Legal compliance requires it. Customer satisfaction demands it.

Who?

The entire team participates. SQA engineers lead activities. Developers, analysts, testers, configuration managers, and management all have quality responsibilities.

When?

Throughout the entire Software Development Life Cycle — from planning and requirements through design, coding, testing, deployment, and maintenance.

How?

Through audits, reviews, inspections, testing, metrics collection, standards enforcement, process improvement (CMMI), and quality frameworks (ISO, IEEE).



Traditional SQA vs. Modern SQA (Agile/DevOps)

How SQA has evolved from waterfall to agile/DevOps environments:

Aspect	Traditional SQA (Waterfall)	Modern SQA (Agile/DevOps)
Timing	End-of-phase quality gates	Continuous quality throughout sprints
Testing approach	Sequential, big-bang testing	Test-first (TDD), continuous testing
SQA team role	Separate, independent group	Embedded in cross-functional teams
Documentation	Comprehensive SQAP upfront	Lightweight, living documents
Defect feedback	Slow — weeks or months	Fast — hours or days
Automation	Limited, manual-heavy	Extensive CI/CD automation
Quality criteria	Defined at start, fixed	Defined per sprint, evolved
Customer involvement	At acceptance testing only	Continuous customer feedback
Process improvement	Post-project retrospective	Sprint retrospectives (every 2 weeks)
Metrics	Phase-end defect counts	Real-time dashboards and velocity
Risk approach	Risk assessment upfront	Continuous risk management
Deployment	Infrequent, big releases	Frequent small releases (CI/CD)

Practice Exercises — Chapter 1

Exercise 1**SQA Plan Draft**

Draft a one-page Software Quality Assurance Plan (SQAP) for a hypothetical student registration system. Include: purpose, standards to follow, reviews to conduct, testing approach, and defect management process.

Exercise 2**Cost of Quality Analysis**

A software project has: 100 hours of code reviews (prevention), 200 hours of testing (appraisal), 50 hours of rework before release (internal failure), and 2 production incidents costing 500 hours (external failure). Calculate total Cost of Quality and the optimal balance.

Exercise 3**CMMI Assessment**

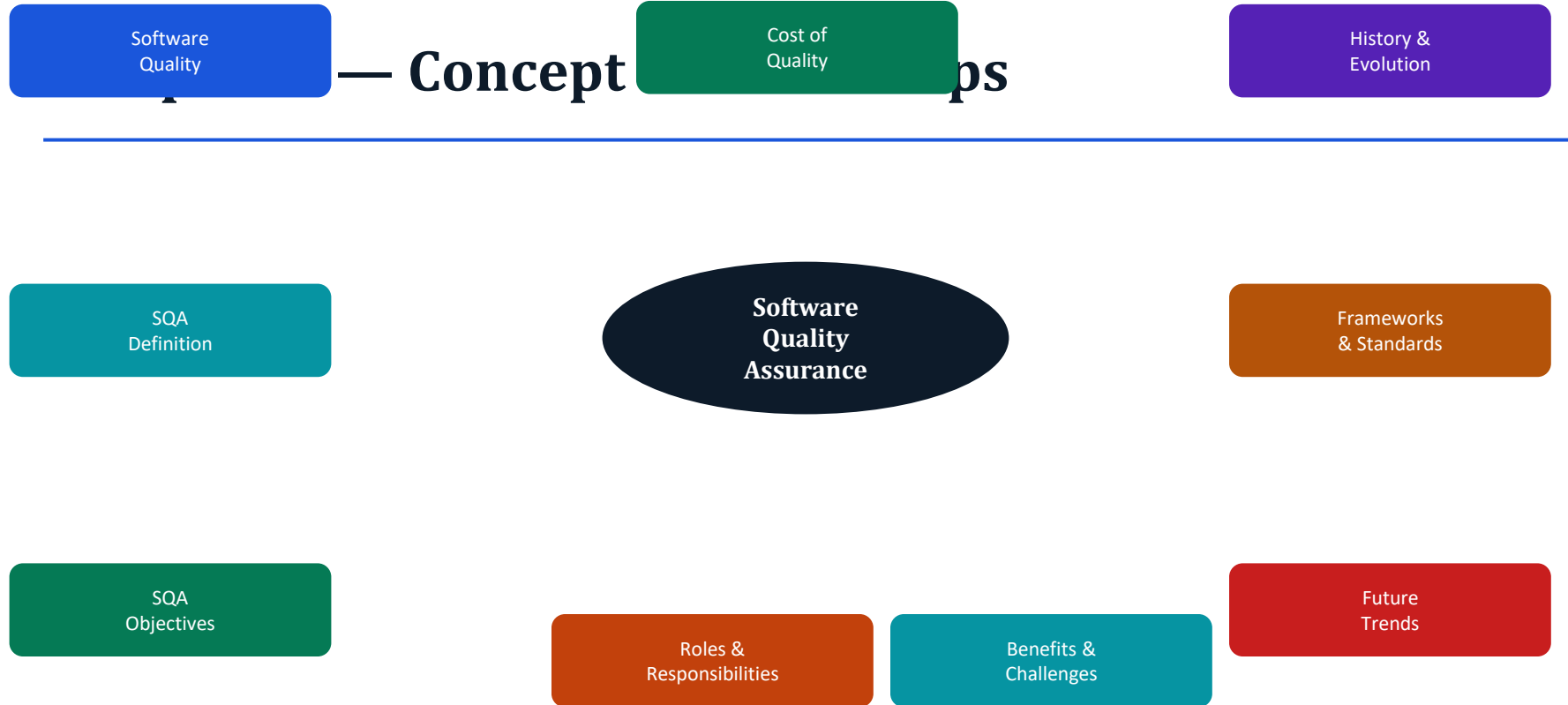
Research your country's largest software company. Based on publicly available information, estimate which CMMI maturity level they likely operate at. Justify your answer with evidence of their processes, certifications, and quality practices.

Exercise 4**Role-Play: SQA Audit**

In pairs, conduct a mock SQA audit. One person plays the SQA auditor, the other plays the development team lead. The auditor must check: coding standards, test coverage, defect tracking, requirements traceability. Document findings and corrective actions.

Exercise 5**SQA Trend Research**

Research one future trend in SQA (AI testing, DevSecOps, shift-left, etc.). Prepare a 5-minute presentation explaining: what it is, why it matters, current adoption, challenges, and how it changes the SQA engineer's role.



SQA Reviews, Audits & Inspections

Three core SQA mechanisms for quality assessment:

Software Reviews

By: Developer team + peers + SQA

When: During all phases

- Walkthrough — informal review by author
- Technical review — team evaluation of product
- Inspection (Fagan) — formal, role-based defect detection
- Management review — business/project status

Software Audits

By: SQA team (independent)

When: Scheduled and event-triggered

- Process audit — checks adherence to defined process
- Product audit — checks product against specification
- Configuration audit — verifies CM baseline integrity
- Post-project audit — captures lessons learned

Software Inspections (Fagan)

By: Moderator, reader, author, inspectors

When: At end of each phase

- Planning — select moderator and inspectors
- Overview — author presents the product
- Preparation — individual review before meeting
- Inspection meeting — find defects, not fix them
- Rework — author corrects defects found
- Follow-up — verify corrections are made

Key SQA Metrics & Measurement

SQA uses quantitative metrics to objectively assess software quality:

Metric	Formula/Definition	Target/Benchmark	Measures
Defect Density	Defects per KLOC (1000 lines of code)	< 1 defect/KLOC (good quality)	Process quality
Defect Removal Efficiency	% of defects removed before release	> 95% (world-class)	Testing effectiveness
Mean Time Between Failures	Average time system runs without failure	Higher = more reliable	Reliability
Test Coverage	% of code covered by tests	> 80% branch coverage	Test thoroughness
Defect Detection Rate	Defects found per test hour	Rising = good; dropping = missed bugs	Testing efficiency
Requirements Traceability	% of requirements with test cases	100% (all requirements tested)	Completeness
Inspection Efficiency	Defects per hour of inspection	1-2 defects/hr (typical)	Review effectiveness
Customer Defect Escape Rate	Defects reported by customers per release	< 0.1 per release (excellent)	Product reliability
Process Compliance Rate	% of activities following defined process	> 90% (good discipline)	Process adherence
Rework Effort Ratio	% of total effort spent on rework	< 10% (well-managed)	Process efficiency